

DEPARTMENT OF COMPUTER SCIENCE



University of
Reading

CS2DA: Data Structures & Algorithms

Heaps

Recap

- Master Theorem — solve a wide range of recurrence equations from divide & conquer algorithms instantly
- Tree traversal
 - In-order, pre-order and post-order traversal
 - Expression trees, Reverse Polish Notation and stack calculators
- Binary Search Trees — $\mathcal{O}(\log n)$ insert/delete under random insertion order
 - Insert is easy. Delete is harder.
- AVL trees — rebalancing guarantees $\mathcal{O}(\log n)$ insert/delete
 - Insert uses $\mathcal{O}(1)$ rotations. Delete uses $\mathcal{O}(\log n)$ rotations.

Today

- More operations on Binary Search Trees
 - Minimum/maximum, size, select, rank, range
- Heap data structure
 - Priority queues
 - Heapsort
 - Treaps

More Tree Operations

Minimum/Maximum

Finding minimum is easy.

Recall subroutine from binary tree deletion:

```
min(n):  
    if n.left == null  
        return n  
    return min(n.left)
```

Just go left as far as possible.

For maximum, just go right as far as possible.

Size

For AVL trees, we added a **height** field to each node, which tracks height of the subtree rooted at the node:

```
n.height = max(n.left.height, n.right.height) + 1
```

In the same way, we can add a **size** field to every tree node, which tracks size of the subtree rooted at the node:

```
n.size = n.left.size + n.right.size + 1
```

This uses $\mathcal{O}(n)$ space and adds $\mathcal{O}(\log n)$ extra time to every operation that adds/removes/moves a node.

Algorithmically, this is simple to maintain (using integer addition/subtraction), although (just as with height) we need to update it every time a node is modified, including on all ancestors of the node.

Select

In quickselect, we can find the k th order statistic by partitioning and following the partition with the target value.

In a binary search tree, each node is effectively a pivot, so just follow the correct partition (left or right side).

The size field tells us how big each partition is.

```
select(n, k):  
    if n == null  
        return null  
    t = n.left.size  
    if t > k  
        return select(n.left, k)  
    else if t < k  
        return select(n.right, k - t - 1)    -- Adjust index sought by size of  
    else                                     -- subtree skipped.  
        return n
```

Rank

We can also do the inverse of size: what **rank** (position) is an element in a search tree?

All elements in skipped left subtrees are smaller than the target.

So sum the size of left subtrees seen when going right to get rank.

```
rank(n, k):  
    if n == null  
        return 0  
    if n.key < k  
        return rank(n.left, k)  
    else if n.key > k  
        return 1 + n.left.size + rank(n.right, k)  
    else  
        return x.left.size
```


Range (of Keys)

We can also iterate over all the keys of a binary search tree in a **range**.

That is, given *low* and *high*, list all keys between *low* and *high*, in ascending order.

Do an in-order tree traversal, but:

- don't recurse left at keys less than or equal to *low*;
- don't recurse right at keys greater than or equal to *high*;
- only output keys between *low* and *high*.

```
range(n, low, high):  
    if n == null  
        return  
    if low < n.key  
        keys(n.left, low, high)  
    if low <= n.key and n.key <= high  
        output n.key  
    if n.key < high  
        keys(x.right, low, high)
```

Priority Queues using Heaps

Priority Queues

Often, we want to find the minimum (or maximum) of a set of elements.

Examples:

- Event-based simulation: Handle next event to occur (smallest time).
- Operating system process scheduling: Schedule task with highest priority.
- Dijkstra's shortest path algorithm: Search from shortest unexplored path.

A **Priority Queue** needs to support the following operations:

- `insert(k,v)` — add the **key** k with **priority** p to the queue
- `min()` — get the **key** with the **lowest priority**
- `del_min()` — get the **key** with the **lowest priority** and delete from the queue

We can do this inefficiently with sorted/unordered arrays, or by periodically sorting.

Heaps: The Heap Property

A **min-heap** is a binary tree satisfying the (minimum) **heap property**:

- Every node is smaller than (or equal to) its children.

This means the smallest key is at the root of the tree.

The tree is *relatively* unconstrained, so updates can be faster and simpler than with a balanced Binary Search Tree.

(We can also have **max-heaps** where every node is greater than its children, so finding and deleting the maximum is fast.)

Heap-Allocated is not Heap Data Structure

We sometimes talk about stack-allocated and heap-allocated memory. Roughly:

- **Stack-allocated:** Local variables, which exist only until a function ends.
- **Heap-allocated:** Memory allocated with `new`, which is garbage-collected when there are no references left (in Java) or manually with `delete` (in C++).

Stack-allocated memory does follow a stack discipline: the most-recently allocated memory is the first to be freed.

Heap-allocated memory has no obvious relationship with heap data structures.

(There is some speculation about whether the term "heap-allocated" derives from some early programming language implementation using a heap data structure to manage blocks of memory of different sizes.)

Implicit Heaps

Recall we want **balanced** binary trees to get $\mathcal{O}(\log n)$ operations.

In a **complete** binary tree:

- all levels except the bottom level are full (so nodes have 2 children);
- the bottom level is filled from left to right.

Heaps are easy to maintain in complete binary tree shape.

Complete binary trees can be efficiently represented by an array.

This is called an **implicit** heap because pointers to children are not stored in memory. Instead, positions of parents and children are calculated with arithmetic.

Implicit Heap Calculations

Store an n -element heap as an $n+1$ -element array A .

(For once, starting at index 1 makes calculations easier!)

- Minimum element of heap: $A[1]$
- Left child of $A[i]$: $A[2*i]$
- Right child of $A[i]$: $A[2*i+1]$
- Parent of $A[i]$: $A[i/2]$

Finding the minimum is $\mathcal{O}(1)$.

Inserting Into a Heap — $\mathcal{O}(\log n)$

Put the new item in the next free space, then let it `swim()` towards the root to restore the heap property.

```
insert(Item x) {
    a[++N] = x;
    swim(N);
}

swim(int k) {
    while (k > 1 && a[k/2] > a[k]) { // stop when k has swum far enough
        exch(k/2, k); // swap k with parent
        k = k/2; // and repeat
    }
}

exch(int i, int j) {
    Item t = a[i]; // use temporary variable to swap
    a[i] = a[j];
    a[j] = t;
}
```


Removing Minimum From a Heap — $\mathcal{O}(\log n)$

Swap the root (minimum) with the last item, then `sink()` it towards the bottom to restore the heap property.

```
del_min() {
    exch(1, N--);
    sink(1, N);
    return a[N+1];
}

sink(int k, int N) {
    while (2*k <= N) {
        int j = 2*k; // j is left child of k
        if (j < N && a[j] > a[j+1]) j++; // j is greatest child of k
        if (a[k] <= a[j]) break; // stop if k has sunk far enough
        exch(k, j); // else swap with j
        k = j; // and repeat
    }
}
```

Changing Priority

In some applications, we need to change an item's priority after insertion.

Example: In Dijkstra's algorithm, if we discover a new, shorter path to a node, we want to continue exploring from there sooner.

If an item's key decreases, let it `swim()`.

If an item's key increases, let it `sink()`.

But how do we find the item?

If changing priority is important, we need to maintain an **index** where we can lookup the current position of an item.

Heap Sort

Priority Queue Sort

We can use a priority queue for sorting:

- Insert all the elements, one at a time, into the queue.
- Repeatedly delete the minimum, adding it to the output.

Exercise: When using a heap for the priority queue, what is the time/space complexity?

Heapsort

Although the obvious priority queue algorithm gives us asymptotically optimal time/space complexity, we can improve the constant factors with heapsort:

- Make a heap from all the elements at once.
- Repeatedly delete the minimum, adding it to the output.

How do we make a heap all at once, and why is it faster?

Make Heap

Starting from the bottom row of the tree and working upwards, `sink()` every element.

For any node, if both children are heaps, sinking the node makes a heap.

In fact, skip the bottom row, as single nodes are already heaps.

Analysis of Make Heap

Recurrence equation for building a heap of height h :

$$T(h) = 2T(h - 1) + \mathcal{O}(h)$$

$$T(1) = \mathcal{O}(1)$$

Which solves to:

$$T(h) = \mathcal{O}(2^h)$$

But $h = \log n$ so:

$$T(\log n) = \mathcal{O}(2^{\log n}) = \mathcal{O}(n)$$

That is, **making a heap for n nodes** (which has height $\log n$) **takes $\mathcal{O}(n)$ time.**

So using Make Heap makes heapsort $\mathcal{O}(n) + \mathcal{O}(n \log n) = \mathcal{O}(n \log n)$.

No improvement in asymptotic complexity, but "constant factors" reduced.

Implementation Details

If we use a max heap and put the deleted items at the end of the array, we can build up a sorted array in-place.

Code:

```
int N = a.length;
for (int k = N/2; k >= 1; k--) {
    sink(k, N);
}
while (N > 1) {
    exch(1, N--);
    sink(1, N);
}
```

where `sink()` has comparisons swapped to implement max heap.

Try it: 7,3,9,8,1,5,4,0

Why use Heap Sort?

Advantages:

- $\mathcal{O}(n \log n)$ guaranteed time without randomness (unlike quicksort)
- $\mathcal{O}(1)$ space (quicksort is $\mathcal{O}(\log n)$, mergesort is $\mathcal{O}(n)$)

Disadvantages:

- Not stable (unlike merge sort)
- Not as fast as quicksort in practice

Treaps

We have already seen how to implement Maps with hash tables and AVL balanced binary trees.

We can use a hybrid data structure called a **treap** (tree/heap) to get a binary search tree with expected $\mathcal{O}(\log n)$ insert/delete.

Unbalanced BSTs have expected $\mathcal{O}(\log n)$ operations if the items are randomly shuffled, but when we use a Map, we often cannot control insertion order.

Idea: Give each item a **random priority**. Build a BST (respecting key ordering) that also respects heap ordering on priorities.

This simulates insertion in order of priority... which is random order.

Explicit Heaps

Although our treaps will be approximately balanced, they will not be complete binary trees.

So we return to using nodes with pointers, as with BSTs.

Insertion

Insert the node as normal for a BST, according to the key.

Give the node a random (32-bit integer) priority.

Rotate it up the treap until the heap priority is satisfied:

- If insertion was in the left subtree, rotate right.
- If insertion was in the right subtree, rotate left.

Recall rotations respect BST key ordering.

Try it: A 9, G 7, H 2, I 4, L 8, M 1, O 6, R 5, T 3

Deletion

(Imagine giving the node to be deleted infinite priority, so it belongs at the bottom of the treap.)

Rotate the node towards the bottom of the treap.

Choose rotate left or right as necessary to preserve heap property:

- If left subtree has higher priority, rotate right.
- If right subtree was higher priority, rotate left.

When the node is a leaf, deletion is easy.

Summary

With one clever data structure we get:

- **Priority queue** with $\mathcal{O}(\log n)$ insert and delete minimum.
- **Heapsort**: In-place sort with guaranteed $\mathcal{O}(n \log n)$ time and $\mathcal{O}(1)$ space.
- **Treaps**: Map with average $\mathcal{O}(\log n)$ insert and delete.

Min-heap property:

- Every node is smaller than its children.

Implementation details:

- Use an array to represent an implicit complete binary tree.
- If priorities in a priority queue can change, we need to add an index.
- Use Make Heap with a max-heap for efficient heapsort.

DEPARTMENT OF COMPUTER SCIENCE



University of
Reading

Next time: Graphs and Finite State Automata